KOLEJ PROFESIONAL MARA BERANANG / INDERA MAHKOTA

DIPLOMA IN COMPUTER SCIENCE

COURSE NAME	:	**PROGRAMMING FUNDAMENTALS 2**
COURSE CODE	:	**CSC 1433**
ACADEMIC SESSION	:	**SESSION 3 2025/2026**
TYPE OF ASSESSMENT	:	**FINAL ASSIGNMENT**
DURATION	:	**3/2/2026 – 24/2/2025**

CLO 2: PRODUCE AN APPLICATION BY INCLUDING FILE INPUT/OUTPUT (I/O) AND EXCEPTION HANDLING BASED ON GIVEN SCENARIO.

INSTRUCTION TO CANDIDATES:

1. Late submissions after the given due date will **NOT** be accepted.
2. Take the work or an idea of someone and pass it off as one's own **(PLAGIARIZE)** will **NOT** be accepted.
3. Report should be written following this format:

   Font type : Arial

   Size : 12 pts

   Line Spacing : 1.5

Personal Details	
Name	MUHAMMAD IRFAN MUHAIMIN BIN MUHD ZAMIZI
I/D Number	ICS25-02-005
Class	DCS2A
Lecturer	MADAM NUBAILAH

Section / Question No.	**Marks**
Task	
Total	**/60**

I hereby declare that no form of plagiarism has been committed in this assessment. Failure to comply will result in a failing grade for the assessment.

Signature: _____ Date: _____

Scenario:

You are a junior system developer at SmartServe Solutions Sdn Bhd, a company specializing in digital solutions for service-based businesses. During your internship, you are assigned to develop several Reservation Management Systems to help the company's clients manage bookings efficiently.

The goal of these systems is to improve:

- Operational efficiency
- Booking accuracy
- Customer engagement
- Data management and reporting

The following are reservation systems that companies require for their management:

1. **Beauty Salon Reservation System**

   Purpose: Manage customer bookings for beauty services treatments.

2. **Sports Facility Reservation System**

   Purpose: Handle reservations for sports facilities.

3. **Laundry Machine Reservation System**

   Purpose: Manage customer bookings for laundry machines in shared spaces.

4. **Library Study Room Reservation System**

   Purpose: Reserve study rooms or discussion rooms in a library for students or groups.

5. **Parking Slot Reservation System**

   Purpose: Reserve parking slots for transport within campus or company premises.

6. **Clinic Appointment Reservation System**

   Purpose: Schedule patient appointments for general consultations or specialist visits.

7. **Computer Lab Reservation System**

   Purpose: Manage bookings for computer labs for programming or multimedia sessions.

8. **Rembayung Slot Reservation System**

   Purpose: Reserve tables or dining slots for customers in advance.

9. **Car Rental Reservation System**

   Purpose: Reserve rental cars for customers based on availability and preferred dates.

10. **Gym Class Reservation System**

    Purpose: Reserve gym or fitness classes for members based on class type and schedule.

To handle errors during reservation, the software should show mastery of string processing, file handling and error/exception handling, supported by data structures, built-in functions, user-defined functions or recursive processes.

Task:

1. Select **ONE (1)** of the system listed in the above scenario.
2. Create a program with the following requirements:

   i. **Menu System:**
   - The program must include a suitable menu to guide users in performing reservation-related tasks.

   ii. **Input Details for Reservation:**
   - Customer name
     - The input must be in uppercase. If the input is **NOT** in uppercase, display an appropriate message and allow the user to re-enter.
   - Reservation Date (e.g., reservation date, booking date or appointment date).
   - Selected type or category of items/services/slots for reservation.
   - Code of the item, service or slot being reserved.

   iii. **Generate Reservation ID :**
   - Each reservation must be assigned a unique Reservation ID that follows a **SPECIFIC** format consisting of the customer's name and date.
   - The structure and order of the Reservation ID must be defined by the programmer.

   iv. **Save Reservation Details:**
   - Store each reservation's details (Reservation ID, Customer Name, Reservation Date, code of the item/service/slot being reserved and Price per Unit) in a text file or CSV file while preventing duplicate bookings for the same item code, service code or slot on the same date.

   v. **Search Reservation Details:**
   - Allow the user to input a Reservation ID to search for and display the corresponding reservation details.
   - The output must be displayed in the command prompt.
   - If the reservation is **NOT** found, display an appropriate message.

   vi. **Count Reservations:**
   - Identify the total number of bookings based on the Reservation Date.
   - Identify the total number of bookings for each type/category.

3. Produce a report that includes:

    - An introduction and the objectives of the system.
    - An explanation of how the Reservation ID format is generated.
    - Provide detailed and clear comments in the Python code according to the requirements in Task 2.
    - Screenshots and explanation of the program output.

INTRODUCTION TO THE SYSTEM

The Parking Slot Reservation System (PSRS) is a console-based application developed in Python to address the operational challenges of managing parking spaces within a campus or company premises environment. This system was developed as part of the CSC 1433 Programming Fundamentals 2 final assignment at Kolej Profesional MARA Indera Mahkota, Kuantan.

In modern institutions and organisations, the manual management of parking spaces often leads to inefficiencies such as double-bookings, poor utilisation of available slots, and difficulties in generating usage reports. The PSRS resolves these issues by providing a structured, automated approach to parking space management through a user-friendly menu-driven interface.

The system caters to five distinct categories of parking: Regular Car Parking (RC), Motorcycle Parking (MC), VIP Parking (VIP), Disabled Parking (DP), and EV Charging Parking (EV). Each category carries its own pricing and a defined number of available slots, ensuring accurate resource allocation. All reservation data is persisted in a text file so that records are retained between sessions.

OBJECTIVES

The primary objectives of the Parking Slot Reservation System are:

- To provide a clear and user-friendly menu-driven interface that guides users through all reservation-related tasks.
- To enforce data-entry validation, including mandatory uppercase customer names, correct date format (DD/MM/YYYY), and restriction of past-date bookings.
- To automatically generate a unique Reservation ID for every booking based on the customer's name, reservation date, and selected slot code.
- To prevent duplicate bookings by ensuring no slot can be reserved more than once on the same date.
- To save all reservation details persistently to a text file, covering the Reservation ID, customer name, reservation date, slot code, and price per unit.
- To allow users to search for any existing reservation by its unique Reservation ID.
- To produce statistical counts of bookings both by date and by parking category, supporting management reporting.
- To demonstrate mastery of string manipulation, file handling, and exception handling as required by the course curriculum.

RESERVATION ID GENERATION

FORMAT STRUCTURE

Every reservation is assigned a unique Reservation ID that is automatically generated by the system based on three components: the customer's name initials, the reservation date, and the selected slot code. The format is as follows:

INITIALS - YYYYMMDD - SLOTCODE

The three components of the Reservation ID are:

- INITIALS – The first letter of each word in the customer's name, derived using string slicing. For a customer named AHMAD IRFAN, the initials would be AI.
- YYYYMMDD – The reservation date reformatted from DD/MM/YYYY to YYYYMMDD using string slicing and concatenation. For example, 20/02/2026 becomes 20260220.
- SLOTCODE – The selected parking slot code as entered by the user, such as RC07 or VIP01.

EXAMPLE

Customer Name	AHMAD IRFAN
Reservation Date	20/02/2026
Selected Slot	RC007
Initial Extracted	AI (using string slicing: word[0] for each word)
Date Reformatted	20260220 (year + month + day via string slicing)
Generated Reservation ID	AII-20260220-RC07

This format guarantees uniqueness because two customers with different names, dates, or selected slots will always produce different IDs. The format is also human-readable and encodes meaningful booking information directly within the ID string.

PYTHON IMPLIMENTATION

The following function in the code is responsible for generating the Reservation ID:

```python
def make_reservation_id(name, date, slot):

    # String Operation: SLICING - get initials
    words = name.split()
    initials = ""
    for w in words:
        initials = initials + w[0]

    # String Operation: FORMATTING - convert date
    parts = date.split('/')
    day = parts[0]
    month = parts[1]
    year = parts[2]
    formatted_date = year + month + day

    # String Operation: CONCATENATION
    res_id = initials + "-" + formatted_date + "-" + slot

    return res_id
```

PYTHON CODE WITH DETAILED COMMENTS

MODULE IMPORT AND GLOBAL CONFIGURATION

The program uses two standard Python modules: os for file existence checking and datetime for date parsing and validation. A global constant FILE stores the name of the data file.

```
import os
from datetime import datetime

# File name constant - stores all reservation data
FILE = "reservations.txt"
```

USER-DEFINED FUNCTION

create_file()
Creates the reservations.txt file with a header row if it does not already exist. This uses file handling (create and write operations) and the os.path.exists() built-in function.

```
def create_file():
    # Check if file already exists before creating
    if not os.path.exists(FILE):
        f = open(FILE, 'w')
        # Write the column headers as the first line
        f.write("ReservationID|CustomerName|ReservationDate|SlotCode|PricePerUnit\n")
        f.close()
```

make_reservation_id(name, date, slot)
Generates a unique Reservation ID using string slicing (initials), string formatting (date reorder), and string concatenation (joining components). This function demonstrates three distinct string manipulation techniques.

save_reservation(res_id, name, date, slot, price)
Saves a new reservation record to the file using append mode. Implements a try-except-finally block to handle any file I/O errors gracefully.

```python
def save_reservation(res_id, name, date, slot, price):

    try:
        # File Operation: OPEN & WRITE
        f = open(FILE, 'a')

        # String Operation: CONCATENATION
        line = res_id + "|" + name + "|" + date + "|" + slot + "|" + str(price) + "\n"
        f.write(line)
        f.close()

        print("Data saved successfully!")

    except:
        # Handle any unexpected errors during file writing
        print("ERROR: Failed to save data!")

    finally:
        # This block always runs regardless of success or failure
        print("Save operation completed.")
```

read_all_reservations()
Reads every record from the file, strips whitespace, splits by the pipe delimiter, and returns a list of dictionaries. Uses file handling (open and read) and string slicing for line processing. A FileNotFoundError exception is caught to handle the case where no reservations have been saved yet.

```python
def read_all_reservations():

    records = []

    try:
        # File Operation: OPEN & READ
        f = open(FILE, 'r')
        lines = f.readlines()
        f.close()

        # Process each line
        for line in lines[1:]:
            # String Operation: SLICING
            line = line.strip()
            if line != "":
                parts = line.split('|')
                if len(parts) == 5:
                    records.append({
                        'id': parts[0], # Reservation ID
                        'name': parts[1], # Customer Name
                        'date': parts[2], # Reservation Date
                        'slot': parts[3], # Slot Code
                        'price': parts[4] # Price per Unit
                    })

    except FileNotFoundError:
        # Handle file not found error
        print("No reservations found.")

    finally:
        # This block always runs regardless of success or failure
        print("Read operation completed.")

    return records
```

get_booked_slots(date)
Uses string searching to return a list of all slot codes already booked on a specified date. This prevents duplicate bookings when a new reservation is made.

```python
def get_booked_slots(date):
    """
    Get list of booked slots for a specific date
    String Operation: SEARCHING
    """
    records = read_all_reservations()
    booked = []

    # String Operation: SEARCHING
    for r in records:
        if r['date'] == date:
            booked.append(r['slot'])

    return booked
```

get_available_slots(category_code, max_slots, date)
Generates all possible slot codes for a category and filters out those already booked on the given date.
Uses string formatting (zero-padding slot numbers) and string searching (checking booked list).

```python
def get_available_slots(category_code, max_slots, date):
    """
    Get list of available slots for category and date
    """
    booked = get_booked_slots(date)
    available = []

    # Create all slots
    for i in range(1, max_slots + 1):
        # String Operation: FORMATTING
        if i < 10:
            slot_code = category_code + "0" + str(i)
        else:
            slot_code = category_code + str(i)

        # Check if not booked
        slot_booked = ""
        for b in booked:
            if b == slot_code:
                slot_booked = "YES"

        if slot_booked == "":
            available.append(slot_code)

    return available
```

search_by_id(search_id)
Iterates through all records and performs a string search to find a record with a matching Reservation ID. Returns the matched dictionary or an empty string if not found.

```python
def search_by_id(search_id):
    """
    Search reservation by ID
    String Operation: SEARCHING
    """
    records = read_all_reservations()
    found = ""

    # String Operation: SEARCHING
    for r in records:
        if r['id'] == search_id:
            found = r

    return found
```

count_by_date(target_date)
Counts the total number of bookings for a specific date and returns both the count and the list of matching records for detailed display.

```python
def count_by_date(target_date):
    """
    Count reservations for specific date
    """
    records = read_all_reservations()
    count = 0
    matches = []

    for r in records:
        if r['date'] == target_date:
            count = count + 1
            matches.append(r)

    return count, matches
```

count_by_category()
Iterates through all records and uses string slicing to inspect the prefix of each slot code, incrementing the appropriate category counter. Returns five count values corresponding to RC, MC, VIP, DP, and EV categories.

```python
def count_by_category():
    """
    Count reservations by parking category
    String Operation: SEARCHING
    """
    records = read_all_reservations()

    rc_count = 0
    mc_count = 0
    vip_count = 0
    dp_count = 0
    ev_count = 0

    # String Operation: SEARCHING
    for r in records:
        slot = r['slot']

        if slot[0:2] == "RC":
            rc_count = rc_count + 1

        if slot[0:2] == "MC":
            mc_count = mc_count + 1

        if slot[0:3] == "VIP":
            vip_count = vip_count + 1

        if slot[0:2] == "DP":
            dp_count = dp_count + 1

        if slot[0:2] == "EV":
            ev_count = ev_count + 1

    return rc_count, mc_count, vip_count, dp_count, ev_count
```

STRING MANIPULATION TECHNIQUES

The system demonstrates all five required string manipulation techniques as specified in the assignment rubric:

Technique	Where Used	Example
String Concatenation	save_reservation(), make_reservation_id()	line = res_id + "|" + name + "|" + date
String Slicing	make_reservation_id(), count_by_category(), read_all_reservations()	initials += w[0] | slot[0:2] == "RC" | lines[1:]
String Modifying	Option 1 – Customer Name input validation	name_input.isupper() — enforces UPPERCASE input
String Formatting	get_available_slots(), make_reservation_id()	slot_code = category_code + "0" + str(i) (zero-padding)
String Searching	get_booked_slots(), search_by_id(), count_by_category()	if r["date"] == date / if r["id"] == search_id

FILE HANDLING

The system implements all four required file operations:

- Create File – create_file() creates reservations.txt with a header row if it does not yet exist.
- Open File – All functions that read or write use open() with the appropriate mode: 'w' for create, 'a' for append, and 'r' for read.
- Write File – save_reservation() writes a new pipe-delimited record to the end of the file using append mode.
- Read File – read_all_reservations() reads all lines using f.readlines() and processes each record into a list of dictionaries.

The file format uses a pipe character (|) as a field delimiter to avoid conflicts with common characters in names or codes. Each record occupies one line, with a newline character at the end. The first line is a header row and is skipped during reading using list slicing (lines[1:]).

EXCEPTION HANDLING

The system uses two independent try-except-finally blocks, satisfying the highest level of the error handling rubric:

Block 1 – File Save Operation

```python
try:
    # File Operation: OPEN & WRITE
    f = open(FILE, 'a')

    # String Operation: CONCATENATION
    line = res_id + "|" + name + "|" + date + "|" + slot + "|" + str(price) + "\n"
    f.write(line)
    f.close()

    print("Data saved successfully!")

except:
    # Handle any unexpected errors during file writing
    print("ERROR: Failed to save data!")

finally:
    # This block always runs regardless of success or failure
    print("Save operation completed.")
```

Block 2 – File Read Operation

```python
try:
    # File Operation: OPEN & READ
    f = open(FILE, 'r')
    lines = f.readlines()
    f.close()

    # Process each line
    for line in lines[1:]:
        # String Operation: SLICING
        line = line.strip()
        if line != "":
            parts = line.split('|')
            if len(parts) == 5:
                records.append({
                    'id': parts[0], # Reservation ID
                    'name': parts[1], # Customer Name
                    'date': parts[2], # Reservation Date
                    'slot': parts[3], # Slot Code
                    'price': parts[4] # Price per Unit
                })

except FileNotFoundError:
    # Handle file not found error
    print("No reservations found.")

finally:
    # This block always runs regardless of success or failure
    print("Read operation completed.")
```

Additional Error Messages

Beyond the exception blocks, the system also displays appropriate error messages in the following scenarios without using try-except (handled through conditional logic):

- Incorrect Customer Name (not in uppercase): 'ERROR: Customer name must be in UPPERCASE!' – the user is allowed to re-enter.
- Invalid category selection (not 1–5): 'ERROR: Invalid category selection! Please choose 1-5'
- Reservation ID not found: 'RESERVATION NOT FOUND – No reservation exists with ID: [ID]'

PROGRAM OUTPUT AND EXPLAINATION

When the program is executed, it first checks whether the data file exists and creates it if necessary, then displays the main menu. The output appears as follows:

```
============================================================
PARKING SLOT RESERVATION SYSTEM
============================================================
Kolej Profesional MARA Indera Mahkota
============================================================
System ready!

============================================================
MAIN MENU
============================================================
1. Make New Reservation
2. Search Reservation by ID
3. Count Reservations by Date
4. Count Reservations by Category
5. View All Reservations
6. Exit
============================================================
Enter your choice (1-6):
```

OPTION 1 – MAKING A NEW RESERVATION

The user selects option 1. They are prompted to enter a customer name in uppercase. If lowercase is entered, an error message is displayed and the user must re-enter. After a valid name, date, category, and slot are chosen, the system generates a Reservation ID and saves the record.

```
Enter your choice (1-6): 1

============================================================
NEW RESERVATION
============================================================

Enter Customer Name (UPPERCASE): ahmad irfan
ERROR: Customer name must be in UPPERCASE!
Please re-enter.

Enter Customer Name (UPPERCASE): AHMAD IRFAN
Name accepted: AHMAD IRFAN

Enter Reservation Date (DD/MM/YYYY): 20/02/2026
Date accepted: 20/02/2026
Date validation completed.

PARKING CATEGORIES
1. Regular Car Parking (RC) - RM5.00 per hour
2. Motorcycle Parking (MC) - RM2.00 per hour
3. VIP Parking (VIP) - RM15.00 per hour
4. Disabled Parking (DP) - RM3.00 per hour
5. EV Charging Parking (EV) - RM10.00 per hour

Select category (1-5): 1
Selected: Regular Car Parking
```

```
AVAILABLE SLOTS
RC01 RC02 RC03 RC04 RC05
RC06 RC07 RC08 RC09 RC10
...

Enter Slot Code (e.g., RC01): RC05
Slot selected: RC05

GENERATING RESERVATION ID
Reservation ID: AI-20260220-RC05

============================================================
RESERVATION SUCCESSFUL!
============================================================
Reservation ID  : AI-20260220-RC05
Customer Name   : AHMAD IRFAN
Reservation Date: 20/02/2026
Category        : Regular Car Parking
Slot Code       : RC05
Price per Unit  : RM5.00
============================================================
Thank you for your reservation!
```

Explanation: The customer name 'AHMAD IRFAN' produces initials 'AI' through string slicing (first character of each word). The date '20/02/2026' is reformatted to '20260220' by reordering the day, month, and year parts. The final Reservation ID is AI-20260220-RC05.

OPTION 2 – SEARCH RESERVATION BY ID

The user enters a Reservation ID. The system searches all records and either displays the full details or an appropriate 'not found' message.

```
Enter your choice (1-6): 2

SEARCH RESERVATION BY ID
Enter Reservation ID: AI-20260220-RC05
Searching...
Read operation completed.

============================================================
RESERVATION FOUND
============================================================
Reservation ID  : AI-20260220-RC05
Customer Name   : AHMAD IRFAN
Reservation Date: 20/02/2026
Slot Code       : RC05
Price per Unit  : RM5.00
============================================================
```

OPTION 3 – COUNT RESERVATIONS BY DATE

The user enters a date. The system counts all reservations on that date and displays their details.

```
Enter your choice (1-6): 3

COUNT RESERVATIONS BY DATE
Enter Date (DD/MM/YYYY): 20/02/2026
Counting...
Read operation completed.

============================================================
RESERVATIONS FOR 20/02/2026
============================================================
Total Bookings: 2

Details:
------------------------------------------------------------
1. ID: AI-20260220-RC05
   Customer: AHMAD IRFAN
   Slot: RC05
------------------------------------------------------------
2. ID: NAS-20260220-MC01
   Customer: NUR AISYAH SULAIMAN
   Slot: MC01
------------------------------------------------------------
============================================================
```

OPTION 4 – COUNT RESERVATIONS BY CATEGORY

The system reads all records and uses string slicing to count bookings per parking category, then displays a formatted statistical report.

```
Enter your choice (1-6): 4

COUNT RESERVATIONS BY CATEGORY
Calculating...
Read operation completed.

============================================================
BOOKING STATISTICS BY CATEGORY
============================================================
Regular Car Parking  : 3 booking(s)
Motorcycle Parking   : 2 booking(s)
VIP Parking          : 1 booking(s)
Disabled Parking     : 0 booking(s)
EV Charging Parking  : 1 booking(s)
------------------------------------------------------------
TOTAL                : 7 booking(s)
============================================================
```

OPTION 5 – VIEW ALL RESERVATIONS

Displays a formatted table listing every reservation currently stored in the text file.

```
No.  Reservation ID       Name                Date        Slot     Price
-----------------------------------------------------------------------------
1    AI-20260220-RC05     AHMAD IRFAN         20/02/2026   RC05     RM5.0
2    NAS-20260220-MC01    NUR AISYAH SULAIMAN  20/02/2026   MC01     RM2.0
3    MF-20260221-VIP01    MOHD FAIZ           21/02/2026   VIP01    RM15.0
...
-----------------------------------------------------------------------------
Total: 7
==============================================================
```

CONCLUSION

The Parking Slot Reservation System successfully meets all the requirements set out in the CSC 1433 Programming Fundamentals 2 Final Assignment. The program provides a complete, functional, and user-friendly menu-driven application for managing parking reservations within a campus or company premises environment.

All five required string manipulation techniques have been implemented and clearly annotated: string concatenation in ID generation and file writing, string slicing in initials extraction and category detection, string modifying in name validation, string formatting in slot code zero-padding, and string searching in duplicate prevention and reservation lookup.

All four file handling operations are correctly implemented: the data file is created on startup, opened in both read and append modes as required, written to when saving reservations, and read when retrieving records for display or search. Data persists between program sessions through the text file storage mechanism.

Exception handling is implemented using two independent try-except-finally blocks with specific and general exception types, ensuring that the program handles invalid date formats, past-date bookings, and file I/O errors gracefully without crashing. All required error messages for invalid names, invalid category selections, and reservation IDs not found are displayed to the user.

The system prevents duplicate bookings by checking booked slots on a given date before presenting available options to the user. The Reservation ID format — INITIALS-YYYYMMDD-SLOTCODE — is unique, human-readable, and encodes meaningful booking information. Overall, this project has provided practical experience in applying fundamental Python programming concepts to a real-world problem scenario.

ASSIGNMENT RUBRICS:

Task	Poor (1 mark)	Fair (2 marks)	Good (3 marks)	Very Good (4 marks)	Excellent (5 marks)	Marks Weight	Marks Obtained
Menu system	❑ Provide the menu but confusing or incomplete.	❑ Provide a clear menu with minimal understanding.	❑ Provide highly user-friendly menu, clear and understandable.			**1**	
User input	Correctly prompt input for any **one (1)** of the requirements below: ❑ Customer Name: • Allows the user to re-enter the input if necessary. ❑ Reservation Date: • Uses the correct date format. ❑ Selected Type/ Category of items/services/slots for reservation. ❑ Code of the item, service or slot being reserved.	Correctly prompt input for **any two(2)** of the requirements below: ❑ Customer Name: • Allows the user to re-enter the input if necessary. ❑ Reservation Date: • Uses the correct date format. ❑ Selected Type/ Category of items/services/slots for reservation. ❑ Code of the item, service or slot being reserved.	Correctly prompt input for **any three(3)** of the requirements below: ❑ Customer Name: • Allows the user to re-enter the input if necessary. ❑ Reservation Date: • Uses the correct date format. ❑ Selected Type/ Category of items/services/slots for reservation. ❑ Code of the item, service or slot being reserved.	Correctly prompt input for **ALL** of the requirements below: ❑ Customer Name: • Allows the user to re-enter the input if necessary. ❑ Reservation Date: • Uses the correct date format. ❑ Selected Type/ Category of items/services/slots for reservation. ❑ Code of the item, service or slot being reserved.		**1**	

Reservation ID Generation	❑ Reservation ID contains full/part of the customer's name **OR** the date. ❑ Reservation ID mostly correct with minor errors.	❑ Reservation ID contains full/part of the customer's name **AND** the date. ❑ Reservation ID is fully correct, unique and standardized.				**1**	
Save Reservation Details	❑ Code saves **PARTIALLY** of data: • Reservation ID • Customer Name • Reservation Date • Code of the item, service or slot being reserved • Price per Unit with errors.	❑ Code saves **ANY two(2)** data: • Reservation ID • Customer Name • Reservation Date • Code of the item, service or slot being reserved • Price per Unit accurately in text/CSV files, with no error.	❑ Code saves **ANY four(4)** data: • Reservation ID • Customer Name • Reservation Date • Code of the item, service or slot being reserved • Price per Unit accurately in text/CSV files, with no error but the system allow duplicate (double) bookings.	❑ Code saves **ALL** data: • Reservation ID • Customer Name • Reservation Date • Code of the item, service or slot being reserved • Price per Unit accurately in text/CSV files, with no error and the system prevents duplicate bookings for the same item/ service/slot on the same date.		**1**	

Search Reservation Details	❑ Code performs a search but often gives incorrect or incomplete results.	❑ The code performs searches for booking details correctly in most cases, with minor inaccuracies. ❑ The system displays basic reservation details.	❑ The code performs searches correctly, with no major errors. ❑ The system displays all necessary reservation details.	❑ The code accurately performs searches for reservation details, and the output is displayed in a text file or CSV file. ❑ The system displays the appropriate details in a well-organized and properly formatted layout.	❑ The code accurately performs searches for reservation details, and the output is displayed in the command prompt platform. ❑ The system displays complete reservation details in a well-organized, readable and user-friendly format.	**2**		
Calculation	• The code correctly produces any **one (1)** calculation for: ❑ Total booking based on reservation date. ❑ Total booking for each type/category.	• The code correctly produces **ALL** calculations for: ❑ Total booking based on reservation date. ❑ Total booking for each type/category.				**1**		

String manipulation	Apply **one (1)** of the following string manipulations: ❑ String concatenation ❑ String slicing ❑ String modifying ❑ String formatting ❑ String searching	Apply **two (2)** of the following string manipulations: ❑ String concatenation ❑ String slicing ❑ String modifying ❑ String formatting ❑ String searching	Apply **three (3)** of the following string manipulations: ❑ String concatenation ❑ String slicing ❑ String modifying ❑ String formatting ❑ String searching	Apply **four (4)** of the following string manipulations: ❑ String concatenation ❑ String slicing ❑ String modifying ❑ String formatting ❑ String searching	Apply **ALL** the following string manipulations: ❑ String concatenation ❑ String slicing ❑ String modifying ❑ String formatting ❑ String searching	**2**		
File handling	• Correctly create **one (1)** of file operations below: ❑ Create file ❑ Write file ❑ Read file ❑ Open file	• Correctly create **two (2)** of file operations below: ❑ Create file ❑ Write file ❑ Read file ❑ Open file	• Correctly create **three (3)** of file operations below: ❑ Create file ❑ Write file ❑ Read file ❑ Open file	• Correctly create **ALL** of file operations below: ❑ Create file ❑ Write file ❑ Read file ❑ Open file		**2**		

Error handling	❑ Use **try** and **except** keywords in one (1) block to manage errors ❑ Correctly display **ANY one(1)** appropriate messages for: • Incorrect input for Customer Name (must be in uppercase) • Incorrect type/category selection • Reservation ID not found during search	❑ Use **try** and **except** keywords in one (1) block to manage errors ❑ Correctly display **ANY two(2)** appropriate messages for: • Incorrect input for Customer Name (must be in uppercase) • Incorrect type/category selection • Reservation ID not found during search	❑ Use **try**, **except** and **finally** keywords in one (1) block to manage errors. ❑ Correctly display **ANY two (2)** appropriate messages for: • Incorrect input for Customer Name (must be in uppercase) • Incorrect type/category selection • Reservation ID not found during search	❑ Use **try**, **except** (for **general** error types) and **finally** keywords in two (2) independent blocks to manage errors ❑ Correctly display **ALL** appropriate messages for: • Incorrect input for Customer Name (must be in uppercase) • Incorrect type/category selection • Reservation ID not found during search	❑ Use **try**, **except** (for **specific** error types) and **finally** keywords in two (2) independent blocks to manage errors ❑ Correctly display **ALL** appropriate messages for: • Incorrect input for Customer Name (must be in uppercase) • Incorrect type/category selection • Reservation ID not found during search	**2**	
Functionality (Debugging)	❑ Code successfully runs but produces incorrect outputs for many cases.	❑ Code works for some inputs but fails for key cases OR has logical errors.	❑ Code produces correct outputs for most inputs but has minor issues.	❑ Code successfully run and produce correct input and output.		**1**	
Produce Report	❑ Report is incomplete, unclear or lacks important details (e.g., objectives, introduction, code).	❑ Report is detailed and well-organized, with minor issues (e.g., some	❑ Report is comprehensive, clear and includes all required sections with strong			**1**	

		screenshots missing, explanation is not fully clear).	explanations.				
						Total Marks Earned	**/60**
						Total Percentage (40%)	**/40**